FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

Semestrální práce z předmětu
Programování v jazyce ANSI C

Digitální steganografie

Autor:
Filip Černý
A21B0100P
cernyf@students.zcu.cz

Vyučující:
Ing. Kamil Ekštein, Ph.D.
kekstain@kiv.zcu.cz

Datum:
19. ledna 2024

Obsah

1 Úvod **3**

2 Zadání **4**

3 Analýza úlohy **6**
3.1 Kompresní algoritmus 6
3.2 Datové struktury 6
3.2.1 Reprezentace binárních dat 7
3.2.2 Reprezentace slovníku LZW algoritmu pro kompresy 7
3.2.3 Reprezentace slovníku LZW algoritmu pro dekompresy 7
3.3 Kontrola porušenosti ukrytých dat 8
3.3.1 Parity byte 8
3.3.2 Kontrolní součet 8
3.3.3 CRC 9
3.3.4 Secure Hash Algorithms 9
3.3.5 MD5 10
3.3.6 Volba 10

4 Popis implementace **11**
4.1 Moduly 11
4.1.1 Hlavní moduly 11
4.1.2 Pomocné moduly 11
4.2 Průběh programu a komunikace modulů 12
4.2.1 Ukrytí dat - interakce modulů 12
4.2.2 Extrakce dat - interakce modulů 12
4.3 Modul: Compression 13
4.3.1 LZW algoritmus 13
4.4 Modul: Image 15
4.4.1 LSB - Nejméně významný bit 15
4.5 Modul: Payload 16
4.6 Pomocný modul: BinaryData - datová struktura pro binární data . . . 16
4.7 Pomocný modul: Dictionary - datové strutury pro dekompresy 17

4.8 Pomocný modul: Trie - datová struktura pro kompresy 17

5 Uživatelská příručka **18**
5.1 Instalace překladače GCC 18
5.2 Instalace překladače Microsoft Visual C/C++ 18
5.3 Instalace libpng 18
5.4 Postup Překladu 18
5.5 Používání programu 19
5.5.1 Příklady použití 19
5.6 Ukončení programu 20
5.6.1 Exit kód 0 20
5.6.2 Exit kód 1 20
5.6.3 Exit kód 2 21
5.6.4 Exit kód 3 22
5.6.5 Exit kód 4 22
5.6.6 Exit kód 5 22
5.6.7 Exit kód 6 22

6 Závěr **24**
6.1 Časy běhu programu 24

Literatura **25**

Seznam obrázků **26**

Kapitola 1

Úvod

Tento dokument popisuje a definuje řešení semestrální práce z předmětu *KIV/PC* na zadání: **Digitální steganografie**. Je zde uvedeno podle jakého zadání byla práce vypracována. Také je zde uvedena analýza úlohy, kde je podrobný rozbor problémů, které v tomto projektu vznikají. A následně popis a implementace řešení těchto problémů. Následně uživatelská příručka, která popisuje ovládání tohoto programu a ukazuje praktické příklady využití.

Kapitola 2

Zadání

Naprogramujte v ANSI C přenositelnou [1] **konzolovou aplikaci**, která ukryje/obnoví soubor s libovolným užitečným obsahem (tzv. *payload*) do/z digitálního snímku (fotografie) ve formátu PNG nebo BMP. Payload bude nejprve zkomprimován a následně uložen do bitmapové reprezentace snímku po jednotlivých bitech, a to tak, že n-tý bit payloadu bude uložen do LSB modrého kanálu n-tého pixelu tak, jak ukazuje (zjednodušeně, bez komprese) obr. 1 Tím je tedy dáno, že jak v případe formátu PNG, tak BMP bude nutné zajistit, že je bitmapa uložena jako sekvence 8-bitových hodnot intenzity červeného, zeleného a modrého kanálu, tj. tzv. 24-bit RGB. Program

se tedy před skrýváním textu do obrázku nejprve přesvědčí, že je obrázek (PNG nebo BMP) v subformátu 24-bit RGB (není-li toto splněno, ukončí se s chybovým hlášením), pak provede kompresi payloadu bezeztrátovou kompresní technikou LZW, ověří, že se zkomprimovaný obsah „vejde"do pixelů obrázku (tj. zda je jich dostatek - pokud ne, opět se ukončí s chybovým hlášením) a nakonec zkomprimovaný obsah uloží naznačeným způsobem do bitmapy, kterou zapíše na disk.

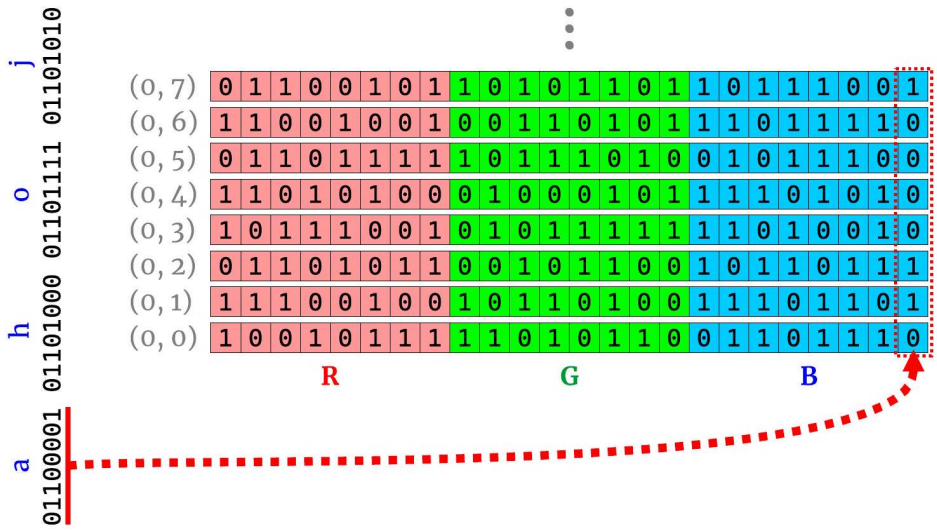
Program se bude spouštět příkazem

`stegim.exe` [2]⟨ *obrázek* [·`png` |. `bmp`] ⟩⟨− *přepínač směru* ⟩⟨ *payload* ⟩

Symbol ⟨*obrázek*⟩ zastupuje první povinný parametr - název souboru s bitmapovým obrázkem ve formátu PNG nebo BMP, subformátu 24-bit RGB, do kterého bude steganograficky ukryt nebo z něj bude naopak extrahován příslušný užitečný obsah (podle toho, jaký bude uveden přepínač směru).

[1] Je třeba, aby bylo možné Váš program přeložit a spustit na PC s operačním prostředím Win32/64 (tj. operační systémy Microsoft Windows NT/2000/XP/Vista/7/8/10/11) a s běžnými distribucemi Linuxu (např. Ubuntu, Debian, Red Hat, atp.). Server, na který budete Vaši práci odevzdávat a který ji otestuje, má nainstalovaný operační systém Debian GNU/Linux 11 (bullseye) s jádrem verze 5.10.0-20-amd64 a s překladačem gcc 10.2.1.

[2] Přípona .exe je povinná i při sestavení v Linuxu, zejm. při automatické kontrole validačním systémem.



Obrázek 2.1: Ukázka způsobu steganografického kódování textu do RGB bitmapy:https://www.kiv.zcu.cz/studies/predmety/pc/data/works/sw2023-02.pdf

Symbol ⟨*-přepinač směru*⟩ pak predstavuje druhý povinný parametr, kterým uživatel určuje, zda se bude payload ze souboru do obrázku zakódovávat nebo se bude z obrázku extrahovat a ukládat na disk. Je-li tento přepínač předán programu ve tvaru '**-h**' (jako ***h**ide*), bude payload specifikovaný třetím povinným parametrem komprimovat a ukrývat do obrázku. Při zadání ve tvaru '-**x**' (jako *e**x**tract*) bude payload vyjímat z bitmapového obrázku, dekomprimovat a ukládat zpět na disk do zvoleného souboru (daného třetím povinným parametrem).

Třetí povinný parametr zastoupený symbolem ⟨ *payload* ⟩ je specifikace souboru na disku, ze kterého program vezme užitečný obsah určený k ukrytí do obrázku nebo do něj naopak užitečný obsah vyjmutý z obrázku uloží.

U obou předávaných souborů (obrázku a payloadu) berte v potaz možnost, že uživatel uvede jak krátké jméno, tak také úplnou specifikaci včetně úplné cesty (a ta může obsahovat i např. mezery).

Podrobnější popis zde

Kapitola 3

Analýza úlohy

Při analýze úlohy vyvstává několik problémů, pro které je potřeba nalézt postup řešení. Nejdříve výběr kompresního alogritmu a poté zvolení vhodných datových struktur pro reprezentaci *LZW slovníku* pro kompresy a dekompresy. Také zvolení datové struktury pro reprezentaci binárních dat. Poté způsob kontroly porušenosti dat.

3.1 Kompresní algoritmus

Pro kompresy dat mohlo být zvoleno mnoho algoritmů jako např. LZ77 nebo gzip, kde každý z nich má své výhody a nevýhody. Pro tento projekt byl zvolen kompresní algoritmus LZW z těchto důvodů:

- Využití tohoto algoritmu bylo definováno v zadání
- Předchozí znalost algoritmu a jeho předešlá implementace
- Vysoká Kompresní Účinnost: LZW dosahuje vysoké míry komprese dat, což je klíčovým kritériem v případě omezeného úložného prostoru nebo přenosové kapacity.
- Univerzálnost: LZW algoritmus je vhodný pro širokou škálu datových typů a formátů. To znamená, že může být použit v různých kontextech bez ohledu na specifické vlastnosti dat.
- Rychlá Implementace: Implementace LZW algoritmu je relativně jednoduchá a efektivní, což usnadňuje jeho rychlé začlenění do projektu.

3.2 Datové struktury

Pro tento projekt bylo důležité zvolit vhodné datové struktury pro reprezentaci binárních dat a pro slovník LZW algoritmu.

3.2.1 Reprezentace binárních dat

Jelikož jazyk C nenabízí datový typ pro reprezentaci binárních dat je potřeba nalézt alternativu. Binární data by se dala reprezentovat jako číslo co nabývá dvou hodnot(0, 1), ale to by zvýšilo nepřehlednost kódu a výsledné pole těchto znaků by bylo velice velké a následná konverze dat do jiných struktur by byla zbytečně náročná. Proto pro tento projekt byla zvoleno dynamické pole s prvky uint8[1] se statickou inkrementací při nedostatku místa. Velikost jednoho prvku byla takto zvolena jelikož je to nejmenší možná datová struktura v jazyce C a reprezentuje jeden byte. Výhody tohoto řešení jsou, že se soubor jednoduše načítá do tohoto pole a při konverze na jiný typ dat lze jednoduše provést konverzi jen pomocí přetypování ukazatele. Pro pole těchto prvků mohl být zvolen *linked list* nebo statické pole s dostatečnou velikostí nebo efektivnější dynamické pole (např. zvyšování o 2x násobek velikosti). Ale bylo zvoleno dynamické pole se zvyšováním velikosti o pevnou hodnotu, jelikož implementace této struktury je jednoduchá a díky tomu není náchylná na vytvoření chyb.

3.2.2 Reprezentace slovníku LZW algoritmu pro kompresy

Slovník LZW algoritmu pro kompresy je reprezentován jako *trie* s 256 potomky [2] kde každý potomek reprezentuje znak, který následně jako hodnotu *trie* obsahuje kód. Při slově delším než jeden znak se *trie* následně rozšiřuje na další potomky. Pro tuto strukturu je potřeba aby byl možný rychlí přístup ke kódu na základě slova a aby struktura neměla statickou velikost jelikož velikost slovníku se liší pro různá data. Alternativy pro toto řešení jsou dynamické pole, linked list, hashovací tabulkou. Dynamické pole je nejjednoduší řešení, ale díky pomalému přístupu k jednotlivým prvkům je nevhodné. Linked list má také pomalý přístup k jednotlivým prvkům. Slovník by mohl být reprezentován hashovací tabulkou, kde by slovo bylo klíč a kód slovníku uložen jako hodnota jedné položky. Řešení hashovací tabulkou je také vhodné řešení pro toto použití, ale díky náročné implementaci byla zvolena trie.

3.2.3 Reprezentace slovníku LZW algoritmu pro dekompresy

Pro názornost a jednoduchost v kódu je slovník LZW při dekompresy reprezentován jako pole kde každý prvek obsahuje hodnotu reprezentovanou jako řetězec a jeho index je kód LZW slovníku. Toto pole je implementováno jako dynamické pole se pevnou inkrementací o konstantu. Alternativy tohoto řešení by byla hash mapa, linked list nebo dynamické pole s jiným způsobem inkrementace. Bylo zvoleno dynamické pole jelikož zajišťuje rychlý přístup k jednotlivým prvkům pomocí indexu, který je stejný jako kód LZW slovníku.

[1] číslo s velikostí 8 bitů

[2] Jelikož LZW algoritmus reprezentuje data jako slova složená z písmen a ty se reprezentují jako 8 bitové číslo, takže existuje 256 možností

3.3 Kontrola porušenosti ukrytých dat

V rámci řešení bylo potřeba zvolit způsob jak zajistit že načtená data z obrázku nebyla poškozena manipulací s obrázkem.

3.3.1 Parity byte

Parita bytu, jako jednoduchá forma kontrolního součtu, nachází své využití v rámci detekce chyb v binárních datech. Paritní bit, buď sudý nebo lichý, je přidán k datům tak, aby vytvořil sudý nebo lichý počet jedniček v celkovém binárním vzoru. Tímto způsobem lze snadno detekovat chyby, zejména jednoduché bitové chyby, při přenosu nebo ukládání dat. Výhodou parity je její jednoduchá implementace a nízká náročnost na zpracování.

Nicméně, existují i některé významné nevýhody spojené s použitím parity pro kontrolu integrity dat. Jednou z hlavních nevýhod je omezená schopnost opravovat chyby. Parita umožňuje pouze detekci chyb, ale neumožňuje identifikovat konkrétní chybové bity nebo je opravit. Navíc, parita je náchylná k detekci pouze lichého počtu jednotlivých bitových chyb; dvě a více chyb, které se vzájemně vyruší, mohou zůstat nezaznamenány.

V dnešní době, s rostoucí složitostí a náročností na spolehlivost přenosu dat, moderní kontrolní součty nebo kódování s opravnou schopností mohou poskytovat lepší možnosti pro zajištění integrity dat v komunikačních nebo ukládacích systémech. Zatímco parita byla historicky důležitým nástrojem, moderní aplikace často volí robustnější a výkonnější metody pro kontrolu integrity dat. [1]

3.3.2 Kontrolní součet

Kontrolní součet, konkrétně doplněný součet (sum complement), slouží k zajištění integrity dat tím, že vytváří kontrolní hodnotu, která je odvozena z aritmetické operace nad daty. Jednou z hlavních výhod je relativně jednoduchá implementace a snadná výpočetní realizace, což umožňuje rychlé ověření integrity dat. V případě doplněného součtu jsou data sečtena, a následně je výsledek complementován. Získaný kontrolní součet je pak připojen k datům a může být použit pro kontrolu integrovanosti při přenosu nebo ukládání.

Nicméně, metoda doplněného součtu má své omezení. Především je citlivá na početní přetečení (overflow), což může způsobit ztrátu informace o chybě. Tato metoda také neumožňuje identifikovat nebo opravit konkrétní chyby, ale pouze indikuje, že chyba nastala. Navíc, při použití pro binární data, může být náchylná k chybám v přenosu bitů, které se vzájemně vyruší a neprojeví se v kontrolním součtu.

V moderních systémech, kde je kladen větší důraz na spolehlivost přenosu dat, se často upřednostňují sofistikovanější kontrolní součty nebo metody kódování s opravnou schopností, které jsou schopny detekovat a někdy i opravit chyby. I přes omezení

je však doplněný součet stále používán v jednodušších aplikacích, kde je dostačující jednoduchost a nízká náročnost na zpracování. [1]

3.3.3 CRC

– Kontrolní součet CRC (Cyclic Redundancy Check) představuje robustnější a sofistikovanější metodu pro kontrolu integrity dat. Na rozdíl od jednoduchých součtů, CRC je schopen identifikovat a detekovat různé typy chyb, včetně chyb ve formě jednotlivých bitů, změn v pořadí bitů nebo náhodných chyb. Je to dáno použitím polynomu, kterým se data násobí při výpočtu kontrolní hodnoty.

Jednou z hlavních výhod CRC je jeho schopnost detekce chyb v rozsáhlejších datech a schopnost poskytnout vyšší úroveň spolehlivosti. Kromě toho CRC umožňuje identifikovat místo a povahu chyb, což usnadňuje diagnostiku problémů při přenosu nebo ukládání dat. Další výhodou je, že CRC je standardizovaný a používaný ve mnoha komunikačních protokolech a souborových formátech.

Omezení metody CRC mohou zahrnovat vyšší nároky na výpočetní výkon v porovnání s jednoduchými součty. Nicméně, moderní výpočetní prostředky jsou obvykle dostatečně výkonné a umožňují efektivní výpočet CRC bez značného zvýšení nákladů. Vzhledem k jeho výkonu a rozšířenému použití se CRC stává preferovanou volbou pro kontrolu integrity dat v komplexnějších systémech. [2]

3.3.4 Secure Hash Algorithms

Secure Hash Algorithms (SHA), zejména SHA-256 nebo SHA-3, představují silné nástroje pro kontrolu integrity dat v důsledku své odolnosti proti kryptoanalýze a vlastnostem odolným vůči kolizím. Tyto algoritmy vytvářejí unikátní hashové hodnoty pro vstupní data, což znamená, že i malá změna ve vstupech vytvoří zcela odlišnou hashovou hodnotu. Tímto způsobem lze s vysokou pravděpodobností detekovat jakoukoli změnu v datech.

Hlavní výhodou SHA oproti jednoduchým kontrolním součtům je jeho schopnost odolávat pokročilým útokům, jako jsou kolizní útoky. Tyto algoritmy jsou navrženy tak, aby byly odolné vůči nelegitimním pokusům o vytvoření dvou různých vstupů se stejnou hashovou hodnotou. SHA algoritmy jsou standardizované a používají se v široké škále bezpečnostních aplikací a protokolů.

Omezení použití SHA mohou zahrnovat nároky na výpočetní výkon, zejména u složitějších variant algoritmů. Nicméně, moderní hardwarová a softwarová řešení obvykle umožňují efektivní výpočty SHA. Další omezení může spočívat v tom, že hashové hodnoty jsou pevně dané délky, což někdy může být nepraktické pro určité aplikace.

Celkově lze říci, že Secure Hash Algorithms poskytují vyšší úroveň bezpečnosti a spolehlivosti při kontrole integrity dat, což je zvláště důležité v citlivých bezpečnostních a kryptografických kontextech. [7]

3.3.5 MD5

MD5 bylo historicky využíváno pro vytváření unikátních hashových hodnot pro vstupní data. Jedná se o algoritmus, který rychle generuje 128bitové hashové hodnoty. Jednou z jeho původních výhod byla rychlost a jednoduchost implementace. MD5 bylo široce používáno k ověření integrity souborů nebo hesel.

Nicméně, v posledních letech bylo objeveno několik zranitelností v návrhu MD5, což způsobilo, že se stalo náchylným k tzv. kolizním útokům. Kolizní útoky umožňují vytvoření dvou různých vstupních sad dat, které generují stejný hash. To zpochybnilo původní zamýšlené použití MD5 pro bezpečnostní účely, jako je ověřování integrity nebo digitální podpisy.

Dnes se nedoporučuje používat MD5 pro bezpečnostní aplikace, kde je nutná odolnost proti kolizím a bezpečnost dat. I přes svou historickou relevanci v rychlém generování hashových hodnot má MD5 v současné době omezené uplatnění v bezpečnostních scénářích. Modernější algoritmy, jako SHA-256 nebo SHA-3, jsou nyní preferovanou volbou, a to zejména tam, kde je bezpečnost klíčovým faktorem. [5]

3.3.6 Volba

Jelikož Parity byte a sum complement nejsou dostačující pro ověření jestli nebyla data výrazně poškozena pro tento projekt bylo zvolen crc přímo crc32 díky předešlé znalosti této checksumy a její implementace a taktéž jelikož díky knihovně *zlib.h* je implementace do programu snadná. SHA a MD5 nebyli zvoleny kvůli náročnosti implementace do tohoto projektu i když jsou to vhodné alternativy.

Kapitola 4

Popis implementace

4.1 Moduly

Hlavní spuštění probíhá v main.c

4.1.1 Hlavní moduly

- Main
- Compression
- Image
- Payload

Moduly si mezi sebou předávají pouze *payloadarray* datový typ, který reprezentuje binární pole.

4.1.2 Pomocné moduly

Ve složce utils jsou vytvořeny pomocné funkce pro tuto aplikaci.

- BinaryData
  - Zde jsou implementovány datové struktury pro práci s binárními daty
- Dictionary
  - Zde jsou implementovány datové struktury pro kompresy pro modul Compression
- Trie

  - Zde je implementována datová struktura trie pro slovník lzw algoritmu pro kompresy

- Logger
  - Zde jsou implementovány funkce pro vypisování chyb a jiných zpráv do konzole

- Memory Management
  - Zde jsou implementovány wrapper funkce pro allokování a dealokování paměti v aplikaci

- Utils
  - Zde jsou implementovány ostatní pomocné funkce pro práci s řetězci a soubory
  - Tento modul vkládá do sebe moduly *logger* a *memory management* a následně se vkládá již jen tento modul do ostatních souborů

4.2 Průběh programu a komunikace modulů

4.2.1 Ukrytí dat - interakce modulů

1. *Main* předá *Payload* jméno souboru s payload daty a *Payload* tato data připraví a vrátí jako binární pole *payloadarray*
   (a) modul *Payload* při přípravě dat využije modul *Compression* kterému předá binární pole *payloadarray* a modul *Compression* vrátí binární pole se zkomprimovanými daty
2. připravené binární pole předá *Main* do modulu *Image* spolu se jménem souboru obrázku a *Image* uloží tyto data do obrázku

4.2.2 Extrakce dat - interakce modulů

1. *Main* předá jméno souboru obrázku se schovanými daty modulu *Image* a ten vrátí binární pole *payloadarray* se schovanými daty extrahovanými z obrázku
2. *Main* toto pole předá modulu *Payload* a ten vrátí původní data modulu *Main* jako řetězec
   (a) modul *Payload* při přípravě dat zavolá modul *Compression* kterému předá řetezec a modul *Compression* mu předá dekomprimovaný řetězec
3. *Main* uloží tato data do výstupního souboru

4.3 Modul: Compression

Tento modul implementuje funkce pro kompresy a dekompresy binárních dat pomocí algoritmu LZW.

4.3.1 LZW algoritmus

LZW algoritmus je unikátní kompresní technikou, která se zaměřuje na identifikaci opakujících se vzorů v datovém proudu a následně na jejich nahrazení krátkými kódy. Klíčovým principem algoritmu je vytváření slovníku, do kterého jsou postupně přidávány nové vzory. Pokud je nějaký vzor v datovém proudu identifikován, je nahrazen odpovídajícím kódem ze slovníku, což výrazně snižuje objem dat. LZW se tímto způsobem adaptuje na strukturu dat a může dosáhnout vysoké míry komprese.

Jednou z klíčových výhod LZW algoritmu je jeho schopnost dosáhnout významné komprese dat bez ztráty informací. Tato metoda je široce používána v kompresních formátech, jako je například GIF pro obrázky nebo TIFF. Další výhodou je rychlá implementace, která umožňuje efektivní kompresi dat s minimálním zpožděním.

Nicméně, je třeba poznamenat, že LZW je bezkontextový algoritmus, což znamená, že každý symbol je komprimován nezávisle na kontextu okolních symbolů. Toto může způsobit, že LZW nemusí být vždy nejúčinnější pro všechny typy dat. [3] [4]

LZW komprese

Implementace komprese je v tomoto projektu podle tohoto pseudokódu.

```
Initialize table with single character strings

     P_INPUT_CHAR = first input character

     WHILE not end of input stream

          C_NEXT_CHAR = next input character

          IF P_INPUT_CHAR + C_NEXT_CHAR is in the string table

            P_INPUT_CHAR = P_INPUT_CHAR + C_NEXT_CHAR

          ELSE

            output the code for INPUT_CHAR

            add P_INPUT_CHAR + C_NEXT_CHAR to the string table

            P_INPUT_CHAR = C_NEXT_CHAR

     END WHILE

    output code for P_INPUT_CHAR
```

LZW dekomprese

Implementace dekomprese je v tomoto projektu podle tohoto pseudokódu.

```
Initialize table with single character strings

OLD = first input code
save to result translation of OLD

WHILE not end of input stream
NEW = next input code
IF NEW is not in the string table
S = translation of OLD
S = S + C
ELSE
S = translation of NEW
output S
C = first character of S
OLD_value + C to the string table
OLD = NEW
END WHILE
```

4.4 Modul: Image

Tento modul implementuje metody pro schování a extrakci binárních dat z modré hodnoty LSB obrázku ve formátu .png a .bmp. Vždy ukládá a načítá data od prvního pixelu po délku připravených dat

4.4.1 LSB - Nejméně významný bit

"Ve výpočetní technice je nejméně významný bit (LSb nebo LSB[1] = least significant bit), který má v celém dvojkovém čísle váhu jedna. Nemusí jít pouze o proměnnou, ale také o registr nebo sběrnici v procesoru nebo v periferii. Jeho hodnota současně určuje lichost nebo sudost čísla.

LSB se také označuje jako bit, který je nejvíce vpravo (right-most bit), což odpovídá běžné konvenci zápisu čísel zleva doprava, od číslice s nejvyšší váhou po číslici s nejnižší váhou. To odpovídá také celočíselným operacím "bitový posuv doleva", který odpovídá násobení dvojkou a "bitový posuv doprava", který odpovídá celočíselnému dělení dvojkou."[6]

V obrázcích tato hodnota vyjadřuje poslední bit modré číselné hodnoty v pixelu.

1	0	0	1	0	1	0	1

Obrázek 4.1: Zápis čísla 149 ve dvojkové soustavě se zvýrazněným nejméně významným bitem https://upload.wikimedia.org/wikipedia/commons/thumb/a/a2/Least_significant_bit.svg/1920px-Least_significant_bit.svg.png

4.5 Modul: Payload

Tento modul zajišťuje za prvé zaobalení dat které mají být schovány se signaturou a kontrolní součet a následně využije modul *Compression* pro kompresy schovaných dat. Tento modul vytváří kontrolní součet na základě zkomprimovaných dat pomocí CRC32.

A za druhé získání ukrytých dat z dat která byla extrahována z obrázku a to odstraněním signatury a kontrolního součtu a následné dekomprese pomocí modulu *Compression*. Tento modul současně kontroluje jestli je signatura validní a jestli je stejný kontrolní součet neboli jestli data nebyla poškozena.

Struktura dat ukládaných do obrázku

- velikost dat v bytech - 4 byty
- signatura - 18 bytů
- kontrolní součet - 4 byty
- zkomprimovaná data pro ukrytí - podle hodnoty velikost dat v bytech

4.6 Pomocný modul: BinaryData - datová struktura pro binární data

Tento modul implementuje strukturu *binarydataarray* která reprezentuje binární data předávaná mezi moduly. Je realizována jako dynamické pole char kde každý element tohoto pole vyjadřuje 8 binárních znaků (jelikož char má velikost 1 byte). Je zde i funkce která vrací jednotlivé bity, které využívají bitové operátory pro získání specifického bytu z elementu. Také je zde funkce pro přidávání jednotlivých bytů do pole která postupně upravuje jednotlivé bity v elementech a přidává elementy podle potřeby pomocí bitových operátorů. Pole je realizováno jako dynamické a v okamžiku že je plné je inkrementováno o konstantu 1000 a realokována paměť.

4.7 Pomocný modul: Dictionary - datové strutury pro dekompresy

Tento modul implementuje datové struktury pro slovník LZW algoritmu při dekompresy. Je to realizováno jako struktura dynamického pole která za elementy obsahuje strukturu která představuje jeden slovníkový zápis a tento slovníkový element je tvořen z kódu (relizováno jako int 16 bitů) a hodnoty (realizováno jako řetězec). V okamžiku je je pole slovníkových zápisů plné zvýší se kapacita pole o konstantu a je realokována paměť.

Pro reprezentaci polí slovníkových hodnot se taktéž používá dynamické pole s zvýšením kapacity o konstantu. Pro reprezentaci pole kódových hodnot se používá dynamické pole se statickou velikostí. Tyto struktury se využívají pro komunikaci mezi funkcemi v modulu *Compression*

4.8 Pomocný modul: Trie - datová struktura pro kompresy

Zde je implementována datová struktura trie. Slovník LZW algoritmu pro kompresy je reprezentován jako *trie* s 256 potomky [1] kde každý potomek reprezentuje znak, který následně jako hodnotu *trie* obsahuje kód. Při slově delším než jeden znak se *trie* následně rozšiřuje na další potomky.

[1] Jelikož LZW algoritmus reprezentuje data jako slova složená z písmen a ty se reprezentují jako 8 bitové číslo, takže existuje 256 možností

Kapitola 5

Uživatelská příručka

Tento program funguje na OS: Win32/64 UNIX/Linux pro tyto systémy jsou připravené makefile

Pro překlad tohoto projektu je potřeba mít nástroj pro překlad jazyka C (gcc, Microsoft Visual C/C++) a knihovnu libpng

5.1 Instalace překladače GCC

Win32/64 https://dev.to/gamegods3/how-to-install-gcc-in-windows-10-the-easier-way-422j

UNIX/Linux https://www.geeksforgeeks.org/how-to-install-gcc-compiler-on-linux/

5.2 Instalace překladače Microsoft Visual C/C++

Win32/64 https://learn.microsoft.com/cs-cz/cpp/windows/latest-supported-vc-redist?view=msvc-170

UNIX/Linux https://www.geeksforgeeks.org/how-to-install-visual-c-on-linux/

5.3 Instalace libpng

http://www.libpng.org/pub/png/libpng.html

5.4 Postup Překladu

1. Nainstalujte překladač GCC nebo Microsoft Visual C/C++ podle vašeho operačního systému

2. Nainstalujte knihovnu libpng
3. Otevřete příkazový řádek na kořenovou složku projektu
4. Přeložte program pomocí příkazu make s jedním ze dvou souborů
   (a) *Makefile* pro GCC překladač
   (b) *Makefile.win*) pro Microsoft Visual C/C++ s *libpng.lib* v adresáři "C:/Program Files(x86)/GnuWin32/lib"
5. Spusťte program s vašimi parametry

5.5 Používání programu

V okamžiku že jste program přeložili můžete program spustit takto
`stegim.exe` ⟨ *obrázek* [·`png` |. `bmp`] ⟩⟨− *přepínač směru* ⟩⟨ *payload* ⟩

Symbol ⟨*obrázek*⟩ zastupuje první povinný parametr - název souboru s bitmapovým obrázkem ve formátu PNG nebo BMP, subformátu 24-bit RGB, do kterého bude steganograficky ukryt nebo z něj bude naopak extrahován příslušný užitečný obsah (podle toho, jaký bude uveden přepínač směru).

Symbol ⟨*-přepinač směru*⟩ pak predstavuje druhý povinný parametr, kterým uživatel určuje, zda se bude payload ze souboru do obrázku zakódovávat nebo se bude z obrázku extrahovat a ukládat na disk. Je-li tento přepínač předán programu ve tvaru '**-h**' (jako ***h**ide*), bude payload specifikovaný třetím povinným parametrem komprimovat a ukrývat do obrázku. Při zadání ve tvaru '-**x**' (jako *e**x**tract*) bude payload vyjímat z bitmapového obrázku, dekomprimovat a ukládat zpět na disk do zvoleného souboru (daného třetím povinným parametrem).

Třetí povinný parametr zastoupený symbolem ⟨ *payload* ⟩ je specifikace souboru na disku, ze kterého program vezme užitečný obsah určený k ukrytí do obrázku nebo do něj naopak užitečný obsah vyjmutý z obrázku uloží.

5.5.1 Příklady použití

Operační systém Windows

Příklad 1: Ukrytí dat

- **Zdrojový a cílový obrázek:** E:\My Work\Test\Holiday 01.png
- **Přepínač směru:** -h
- **Soubor ukrývaný do obrázku:** secret\text.txt

```
stegim.exe "E:\My Work\Test\Holiday 01.png"-h secret\text.txt
```

Příklad 2: Extrakce dat

- **Zdrojový a cílový obrázek:** E:\Download\Beach.bmp
- **Přepínač směru:** -x
- **Soubor ukrývaný do obrázku:** E:\Documents\Tajná zpráva.txt

```
stegim.exe "E:\Download\Beach.bmp"-x "E:\Documents\Tajná zpráva.txt"
```

Operační systém GNU/Linux, FreeBSD, macOS, atp

Příklad 3: Ukrytí dat

- **Zdrojový a cílový obrázek:** /home/user/images/img001.png
- **Přepínač směru:** -h
- **Soubor ukrývaný do obrázku:** /home/user/doc/secret.txt

```
stegim.exe /home/user/images/img001.png -h /home/user/doc/secret.txt
```

5.6 Ukončení programu

5.6.1 Exit kód 0

Program úspěšně skončil.

Výpis:

```
Hiding completed.
exit code 0
```

```
Extraction completed.
exit code 0
```

5.6.2 Exit kód 1

Vyjadřuje nesprávně zadané či chybějící parametry na příkazové řádce (také neexistující vstupní/výstupní soubory)

Výpis:

```
Wrong number of arguments - needed 3 arguments
You can use this program for hiding data in image or extract hidden
↪  data from image.
Hiding: stegim.exe <image_to_hide_to_filepath> -h
↪  <hide_payload_filepath>
Extraction: stegim.exe <input_image_to_extract_from_filepath> -x
↪  <output_payload_filepath>
exit code 1
```

```
Invalid direction flag. Use -h for hiding or -x for extraction.
You can use this program for hiding data in image or extract hidden
↪  data from image.
Hiding: stegim.exe <image_to_hide_to_filepath> -h
↪  <hide_payload_filepath>
Extraction: stegim.exe <input_image_to_extract_from_filepath> -x
↪  <output_payload_filepath>
exit code 1
```

```
Unable to hide the payload into the image.
Files doesnt exist or cannot be open.
exit code 1
```

```
Unable to extract the payload from the image.
Files doesnt exist or cannot be open.
exit code 1
```

5.6.3 Exit kód 2

Vyjadřuje nevhodný formát obrázku (tj. není to PNG či BMP, 24-bit/pixel, RGB).

Výpis:

```
Unable to hide the payload into the image.
Input file is not BMP or PNG 24 bit color.
exit code 2
```

```
Unable to extract the payload from the image.
Input file is not BMP or PNG 24 bit color.
exit code 2
```

5.6.4 Exit kód 3

Vyjadřuje že obrázek není dostatečně velký pro ukrytí zadaného payloadu a potřebných doprovodných informací.

Výpis:

```
Unable to hide the payload into the image.
Payload is too big for image.
exit code 3
```

5.6.5 Exit kód 4

Vyjadřuje že obrázek určený k extrakci payloadu neobsahuje žádný ukrytý ob- sah.

Výpis:

```
Unable to extract the payload from the image.
Image doesnt have payload inside.
exit code 4
```

5.6.6 Exit kód 5

Vyjadřuje že ukrytý obsah byl poškozen, neodpovídá kontrolní součet obsahu či nelze obsah korektně dekomprimovat

Výpis:

```
Unable to extract the payload from the image.
Hidden file in image was corrupted.
exit code 5
```

5.6.7 Exit kód 6

Vyjadřuje jakoukoli jinou chybu (alokace, špatné argumenty ve funkci)

Výpis:

```
Unable to hide the payload into the image.
Other error.
exit code 6
```

```
Unable to extract the payload from the image.
Other error.
exit code 6
```

Kapitola 6

Závěr

Zadání projektu bylo splněno ve všech bodech. Program dokáže ukrýt data do obrázku typu BMP nebo PNG a následně i tyto data extrahovat z obrázku, který je obsahuje.

Pro zlepšení rychlosti programu by pomohli vhodnější datové struktury jako např. HashMapa pro slovník LZW. Také by pomohlo efektivnější čtení souboru obrázku a načítání jeho dat.

Problém při vývoji projektu nastal při čtení BMP souboru jelikož bylo špatně definované jeho čtení a rozdělení jeho částí. Problém byl i s nalezením všech úniků paměti a zajištění aby byla veškerá paměť uvolněna.

6.1 Časy běhu programu

Vstupní data

- BMP obrázek: 29,5 kB
- PNG obrázek: 512 B
- Payload soubor: 32 B

Výsledky

- Ukrytí do BMP souboru: 12.591 ms
- Ukrytí do PNG souboru: 11.156 ms
- Extrakce z BMP souboru: 4.872 ms
- Extrakce z PNG souboru: 5.570 ms

Literatura

[1] Checksum - wikipedia. https://en.wikipedia.org/wiki/Checksum. (Accessed on 01/14/2024).

[2] Cyclic redundancy check - wikipedia. https://en.wikipedia.org/wiki/Cyclic_redundancy_check. (Accessed on 01/14/2024).

[3] Lempel–ziv–welch - wikipedia. https://en.wikipedia.org/wiki/Lempel%E2%80%93Ziv%E2%80%93Welch. (Accessed on 01/14/2024).

[4] Lzw (lempel–ziv–welch) compression technique - geeksforgeeks. https://www.geeksforgeeks.org/lzw-lempel-ziv-welch-compression-technique/. (Accessed on 01/14/2024).

[5] Md5 - wikipedia. https://en.wikipedia.org/wiki/MD5. (Accessed on 01/14/2024).

[6] Nejméně významný bit – wikipedie. https://cs.wikipedia.org/wiki/Nejm%C3%A9n%C4%9B_v%C3%BDznamn%C3%BD_bit. (Accessed on 01/14/2024).

[7] Secure hash algorithms - wikipedia. https://en.wikipedia.org/wiki/Secure_Hash_Algorithms. (Accessed on 01/14/2024).

Seznam obrázků

2.1 Ukázka způsobu steganografického kódování textu do RGB bitmapy:https://www.kiv.zcu.cz/studies/predmety/pc/data/works/sw2023-02.pdf 5

4.1 Zápis čísla 149 ve dvojkové soustavě se zvýrazněným nejméně významným bitem https://upload.wikimedia.org/wikipedia/commons/thumb/a/a2/Least_significant_bit.svg/1920px-Least_significant_bit.svg.png 16